CLOUD & VIRTUALISATION | TEK-UP UNIVERSITY

Lab 06

Creating and deploying a Docker image for a web application

Student
Assil Mhadhbi

Class
Ing1, Groupe C (evening course)

Date
7 November 2026

About this report

In this lab I first ran Apache directly on my Red Hat virtual machine, to see what a web server does without Docker. Then I put the same site in a container and built three versions of the image, to understand why a container stops or keeps running. After that I reached the container in three different ways (its own IP address, a published port, and the host network), and finally I containerised a small Flask application written in Python.

Two things differ from the handout. First, the handout is written for RHEL 9, but my VM runs RHEL 10; this changes nothing for Apache, and the container images use UBI 9 anyway. Second, I made a number of small mistakes in commands and files, so several parts are in boxes called Problem I ran into. I kept them because each one taught me how Docker reads a Dockerfile or a command.

The report follows the order of the handout and every part was completed. The answers to the questions are in section 7.

1. My setup

Item	What I used
Virtual machine	Red Hat Enterprise Linux 10 in VirtualBox (NAT network, address 10.0.2.15), user darkdiva
Web server on the VM	Apache httpd 2.4.63, installed with dnf
Docker	Docker Engine 29.9.1
Base image	registry.access.redhat.com/ubi9/ubi (UBI 9, free, no account needed)
Working folders	~/webserver_apache for the Apache images and ~/webserver_python for the Flask application
Ports used	80 (host network test), 3000 (published port), 5000 (Flask)

2. Part 1: Apache on the VM, without Docker

2.1 Installing and starting Apache. I installed Apache with `sudo dnf install -y httpd` (11 packages, httpd 2.4.63) and started it with `sudo systemctl enable --now httpd`. The service is `enabled` and `active (running)`, and the log says `Server configured, listening on: port 80`.

```
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :                                                                                1/1
  Installing       : apr-1.7.5-3.el10.x86_64                                                        1/11
  Installing       : apr-util-lmdb-1.6.3-23.el10_2.1.x86_64                                         2/11
  Installing       : apr-util-openssl-1.6.3-23.el10_2.1.x86_64                                      3/11
  Installing       : apr-util-1.6.3-23.el10_2.1.x86_64                                              4/11
  Installing       : httpd-tools-2.4.63-13.el10_2.8.x86_64                                          5/11
  Running scriptlet: httpd-filesystem-2.4.63-13.el10_2.8.noarch                                     6/11
  Installing       : httpd-filesystem-2.4.63-13.el10_2.8.noarch                                     6/11
  Installing       : httpd-core-2.4.63-13.el10_2.8.x86_64                                           7/11
  Installing       : mod_http2-2.0.29-4.el10_2.2.x86_64                                             8/11
  Installing       : mod_lua-2.4.63-13.el10_2.8.x86_64                                              9/11
  Installing       : redhat-logos-httpd-100.3-2.el10.noarch                                        10/11
  Installing       : httpd-2.4.63-13.el10_2.8.x86_64                                               11/11
  Running scriptlet: httpd-2.4.63-13.el10_2.8.x86_64                                               11/11
Installed products updated.

Installed:
  apr-1.7.5-3.el10.x86_64                          apr-util-1.6.3-23.el10_2.1.x86_64
  apr-util-lmdb-1.6.3-23.el10_2.1.x86_64           apr-util-openssl-1.6.3-23.el10_2.1.x86_64
  httpd-2.4.63-13.el10_2.8.x86_64                  httpd-core-2.4.63-13.el10_2.8.x86_64
  httpd-filesystem-2.4.63-13.el10_2.8.noarch       httpd-tools-2.4.63-13.el10_2.8.x86_64
  mod_http2-2.0.29-4.el10_2.2.x86_64               mod_lua-2.4.63-13.el10_2.8.x86_64
  redhat-logos-httpd-100.3-2.el10.noarch

Complete!
darkdiva@localhost:~$ sudo systemctl enable --now httpd
Created symlink '/etc/systemd/system/multi-user.target.wants/httpd.service' → '/usr/lib/systemd/system/httpd.service'
darkdiva@localhost:~$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
     Active: active (running) since Sat 2026-11-07 11:13:26 CET; 14s ago
 Invocation: 7da4c4d7751940ed81fe1f4dcd417d23
       Docs: man:httpd.service(8)
   Main PID: 5871 (httpd)
     Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec:   0 B/sec"
      Tasks: 177 (limit: 22988)
     Memory: 13.6M (peak: 13.8M)
        CPU: 107ms
     CGroup: /system.slice/httpd.service
             ├─5871 /usr/sbin/httpd -DFOREGROUND
             ├─5877 /usr/sbin/httpd -DFOREGROUND
             ├─5878 /usr/sbin/httpd -DFOREGROUND
             ├─5879 /usr/sbin/httpd -DFOREGROUND
lines 1-15...skipping...
● httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
     Active: active (running) since Sat 2026-11-07 11:13:26 CET; 14s ago
 Invocation: 7da4c4d7751940ed81fe1f4dcd417d23
       Docs: man:httpd.service(8)
```

Figure 1. End of the installation, and httpd active (running).

2.2 What Apache shows with an empty site. I opened HTTP in the firewall with `firewall-cmd --permanent --add-service=http` and `--reload`, then tested with `curl -I http://localhost`. The answer is **`HTTP/1.1 403 Forbidden`**, with a page of 5,909 bytes. The folder `/var/www/html` is empty (`total 0`). The reason is in `/etc/httpd/conf.d/welcome.conf`: for the root address, it sets `Options -Indexes` (Apache refuses to list a folder) and `ErrorDocument 403 /.noindex.html`, and an `Alias` maps that file to the Red Hat welcome page in `/usr/share/httpd/noindex`. So an empty site gives a 403 code, and Apache answers with the Welcome page.

```
darkdiva@localhost:~$ sudo firewall-cmd --permanent --add-service=http
success
darkdiva@localhost:~$ sudo firewall-cmd --reload
success
darkdiva@localhost:~$ curl -I http://localhost
HTTP/1.1 403 Forbidden
Date: Sat, 07 Nov 2026 10:18:15 GMT
Server: Apache/2.4.63 (Red Hat Enterprise Linux)
Last-Modified: Wed, 10 Dec 2025 14:42:03 GMT
ETag: "1715-6459a0812e4c0"
Accept-Ranges: bytes
Content-Length: 5909
Content-Type: text/html; charset=UTF-8

darkdiva@localhost:~$ ls -l /var/www/html
total 0
darkdiva@localhost:~$ cat /etc/httpd/conf.d/welcome.conf
#
# This configuration file enables the default "Welcome" page if there
# is no default index page present for the root URL.  To disable the
# Welcome page, comment out all the lines below.
#
# NOTE: if this file is removed, it will be restored on upgrades.
#
<LocationMatch "^/+$">
    Options -Indexes
    ErrorDocument 403 /.noindex.html
</LocationMatch>

<Directory /usr/share/httpd/noindex>
    AllowOverride None
    Require all granted
</Directory>

Alias /.noindex.html /usr/share/httpd/noindex/index.html
Alias /poweredby.png /usr/share/httpd/icons/apache_pb3.png
Alias /system_noindex_logo.png /usr/share/httpd/icons/system_noindex_logo.png
darkdiva@localhost:~$
```

Figure 2. Firewall rule, the 403 answer, the empty folder and welcome.conf.

2.3 Adding my own page. I created `/var/www/html/index.html` with `sudo tee`. Now `curl -I` returns **`HTTP/1.1 200 OK`** (79 bytes) and `curl` prints my page.

```
darkdiva@localhost:~$ sudo tee /var/www/html/index.html << 'EOF'
> <!DOCTYPE html>
> <html>
>    <body><h1>Hello From Docker LAB!</h1></body>
> </html>
> EOF
[sudo] password for darkdiva:
<!DOCTYPE html>
<html>
   <body><h1>Hello From Docker LAB!</h1></body>
</html>
darkdiva@localhost:~$
```

Figure 3. Creating index.html in /var/www/html.

```
darkdiva@localhost:~$ curl -I http://localhost
HTTP/1.1 200 OK
Date: Sat, 07 Nov 2026 10:25:12 GMT
Server: Apache/2.4.63 (Red Hat Enterprise Linux)
Last-Modified: Sat, 07 Nov 2026 10:24:27 GMT
ETag: "4f-65fad2228fe86"
Accept-Ranges: bytes
Content-Length: 79
Content-Type: text/html; charset=UTF-8

darkdiva@localhost:~$ curl http://localhost
<!DOCTYPE html>
<html>
   <body><h1>Hello From Docker LAB!</h1></body>
</html>
darkdiva@localhost:~$ ls -l /var/www/html
total 4
-rw-r--r--. 1 root root 79 Nov  7 11:24 index.html
darkdiva@localhost:~$
```

Figure 4. The code is now 200 OK and the page is served.

2.4 Stopping Apache. Before using containers I stopped Apache with `sudo systemctl disable --now httpd`, because later the containers need port 80. The status is `inactive (dead)` and `disabled`.

```
darkdiva@localhost:~$ sudo systemctl disable --now httpd
Removed '/etc/systemd/system/multi-user.target.wants/httpd.service'.
darkdiva@localhost:~$ sudo systemctl status httpd --no-pager
○ httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
     Active: inactive (dead)
       Docs: man:httpd.service(8)

Nov 07 11:13:26 localhost.localdomain systemd[1]: Starting httpd.service - The Apache HTTP Server...
Nov 07 11:13:26 localhost.localdomain (httpd)[5871]: httpd.service: Referenced but unset environment variable evaluates to a… OPTIONS
Nov 07 11:13:26 localhost.localdomain httpd[5871]: AH00558: httpd: Could not reliably determine the server's fully qualified… message
Nov 07 11:13:26 localhost.localdomain httpd[5871]: Server configured, listening on: port 80
Nov 07 11:13:26 localhost.localdomain systemd[1]: Started httpd.service - The Apache HTTP Server.
Nov 07 11:27:32 localhost.localdomain systemd[1]: Stopping httpd.service - The Apache HTTP Server...
Nov 07 11:27:33 localhost.localdomain systemd[1]: httpd.service: Deactivated successfully.
Nov 07 11:27:33 localhost.localdomain systemd[1]: Stopped httpd.service - The Apache HTTP Server.
Hint: Some lines were ellipsized, use -l to show in full.
darkdiva@localhost:~$
```

Figure 5. Apache stopped and disabled on the VM.

3. Part 2: the Apache image on UBI 9

I created `~/webserver_apache` with the same `index.html`. The three versions of the image use the same base and differ only in the last lines.

```
darkdiva@localhost:~$ mkdir -p ~/webserver_apache && cd ~/webserver_apache
darkdiva@localhost:~/webserver_apache$ cat > index.html << 'EOF'
> '
<!DOCTYPE html>
<html>
   <body><h1>Hello From Docker LAB!</h1></body>
</html>
EOF
darkdiva@localhost:~/webserver_apache$
```

Figure 6. The working folder and index.html.

3.1 Version 1: Apache launched in the background. This is the Dockerfile:

```
FROM registry.access.redhat.com/ubi9/ubi
#Installer Apache et vider le cache dnf
RUN dnf -y install httpd && dnf clean all
#Répertoire du site
WORKDIR /var/www/html
COPY index.html .
#Port de l'application
EXPOSE 80
#Démarrer Apache
CMD ["httpd", "-k", "start"]
```

> **Problem I ran into**
>
> My first `docker build` failed with `COPY requires at least two arguments, but only one was provided. Destination could not be determined.` I had written `COPY index.html` without the destination. `COPY` needs a source and a destination, and the final dot means the current `WORKDIR`, which is `/var/www/html`. Because the build failed, the image `webserver:version1` was never created, so my next `docker run` tried to download it from Docker Hub and answered `pull access denied`. I corrected the line with `sed` and checked it with `grep`.

```
darkdiva@localhost:~/webserver_apache$ cat > Dockerfile << 'EOF'
FROM registry.access.redhat.com/ubi9/ubi
#Installer Apache et vider le cache dnf
RUN dnf -y install httpd && dnf clean all
#Répertoire du site
WORKDIR /var/www/html
COPY index.html
#Port de l'application
EXPOSE 80
#Démarrer Apache
CMD ["httpd", "-k", "start"]
EOF
darkdiva@localhost:~/webserver_apache$ cat Dockerfile
FROM registry.access.redhat.com/ubi9/ubi
#Installer Apache et vider le cache dnf
RUN dnf -y install httpd && dnf clean all
#Répertoire du site
WORKDIR /var/www/html
COPY index.html
#Port de l'application
EXPOSE 80
#Démarrer Apache
CMD ["httpd", "-k", "start"]
darkdiva@localhost:~/webserver_apache$
```

Figure 7. The first version of the Dockerfile with the missing destination.

```
darkdiva@localhost:~/webserver_apache$ docker build -t webserver:version1 .
[+] Building 0.2s (1/1) FINISHED                                                                                    docker:default
 => [internal] load build definition from Dockerfile                                                                       0.1s
 => => transferring dockerfile: 362B                                                                                       0.0s
Dockerfile:6
--------------------
   4 |     #Répertoire du site
   5 |     WORKDIR /var/www/html
   6 | >>> COPY index.html
   7 |     #Port de l'application
   8 |     EXPOSE 80
--------------------
ERROR: failed to build: failed to solve: dockerfile parse error on line 6: COPY requires at least two arguments, but only one was pro
vided. Destination could not be determined
darkdiva@localhost:~/webserver_apache$ docker images
                                                                                                    i Info →  U In Use
IMAGE                                          ID             DISK USAGE   CONTENT SIZE   EXTRA
centos:centos7                                 be65f488b776       299MB         76.1MB
darkdiva/hello:2.0                             14b526799968       108kB         30.1kB
darkdiva/monsite:1.1                           765f8c3bb185      91.3MB         26.1MB
hello-world:latest                             11a3eb1597e8      21.8kB         9.49kB    U
hello:version1                                 906a4946a733       378MB          101MB
hello:version3                                 35d273252cba       502MB          149MB
localhost:5000/hello:2.0                       14b526799968       108kB         30.1kB
localhost:5000/monsite:1.0                     2a2565c044ee      91.3MB         26.1MB
localhost:5000/monsite:1.1                     765f8c3bb185      91.3MB         26.1MB
mon-site:1.0                                   1b3815fadac1      91.3MB         26.1MB
nginx:1.30.6-alpine                            0e52bd054126      92.1MB           27MB
nginx:stable-alpine                            0e52bd054126      92.1MB           27MB
python-hello:full                              69a24d6d26d2      1.59GB          412MB
python-hello:slim                              62d89bcb6014       174MB         43.3MB
registry.access.redhat.com/ubi9/ubi:latest     16e9e43d99e1       309MB         80.5MB
registry:2                                     a3d8aaa63ed8      36.5MB         10.3MB
ubuntu:24.04                                   fe81560990fa       117MB         31.7MB
ubuntu:latest                                  c098726dd9b9       160MB         45.9MB
darkdiva@localhost:~/webserver_apache$ docker run --name webcontainer1 webserver:version1
Unable to find image 'webserver:version1' locally
docker: Error response from daemon: pull access denied for webserver, repository does not exist or may require 'docker login'

Run 'docker run --help' for more information
darkdiva@localhost:~/webserver_apache$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
darkdiva@localhost:~/webserver_apache$ docker ps -a
CONTAINER ID   IMAGE         COMMAND    CREATED       STATUS                     PORTS     NAMES
f7795f10d0ab   hello-world   "/hello"   2 weeks ago   Exited (0) 2 weeks ago             naughty_borg
darkdiva@localhost:~/webserver_apache$
```

Figure 8. The build error, and the run that could not find the image.

After the fix, the build passed (14.8 s) and the image is 345 MB. Then `docker run --name webcontainer1 webserver:version1` printed a warning and **returned to my prompt immediately**. `docker ps` is empty, and `docker ps -a` shows `webcontainer1` as `Exited (0)` a few seconds after its creation, with the command `httpd -k start`.

```
darkdiva@localhost:~/webserver_apache$ sed -i 's/^COPY index.html$/COPY index.html ./' Dockerfile
darkdiva@localhost:~/webserver_apache$ grep -n COPY Dockerfile
6:COPY index.html .
darkdiva@localhost:~/webserver_apache$ docker build -t webserver:version1 .
[+] Building 14.8s (9/9) FINISHED                                                                                    docker:default
 => [internal] load build definition from Dockerfile                                                                          0.0s
 => => transferring dockerfile: 364B                                                                                          0.0s
 => [internal] load metadata for registry.access.redhat.com/ubi9/ubi:latest                                                   0.1s
 => [internal] load .dockerignore                                                                                             0.1s
 => => transferring context: 2B                                                                                               0.0s
 => [1/4] FROM registry.access.redhat.com/ubi9/ubi:latest@sha256:16e9e43d99e181463513cc84a514859b1abb9460f1c4d9e208da4b4e51cd5  4.3s
 => => resolve registry.access.redhat.com/ubi9/ubi:latest@sha256:16e9e43d99e181463513cc84a514859b1abb9460f1c4d9e208da4b4e51cd5  0.1s
 => => extracting sha256:dd4cdd4b0597312ef0bc0be7a2a004d6b50648d4e0e30a28daf0db92b2083c0e                                     4.1s
 => [internal] load build context                                                                                             0.1s
 => => transferring context: 177B                                                                                             0.0s
 => [2/4] RUN dnf -y install httpd && dnf clean all                                                                           8.1s
 => [3/4] WORKDIR /var/www/html                                                                                               0.1s
 => [4/4] COPY index.html .                                                                                                   0.1s
 => exporting to image                                                                                                        1.7s
 => => exporting layers                                                                                                       1.0s
 => => exporting manifest sha256:cabec1df3cf1a20deba2399507ebce26f63f173c8bde7f26efb23504fae62322                             0.0s
 => => exporting config sha256:70e15c7160740242bca4e04a40793bd00a2a4bd8634f8367e78840b950546871                               0.0s
 => => exporting attestation manifest sha256:a4436529c46e8dafc48c1981c5bf40cee0359d3a97505373ac7eb445ee9877f7                 0.0s
 => => exporting manifest list sha256:10116556f363937c8287723a5987d79ccfbb9d3a195340e7867b5b900506e907                        0.0s
 => => naming to docker.io/library/webserver:version1                                                                         0.0s
 => => unpacking to docker.io/library/webserver:version1                                                                      0.5s
darkdiva@localhost:~/webserver_apache$ docker images webserver
                                                                                                  i Info →  U In Use
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
webserver:version1   10116556f363       345MB         87.5MB
darkdiva@localhost:~/webserver_apache$ docker run --name webcontainer1 webserver:version1
AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.17.0.2. Set the 'ServerName' directi
ve globally to suppress this message
darkdiva@localhost:~/webserver_apache$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
darkdiva@localhost:~/webserver_apache$ docker ps -a
CONTAINER ID   IMAGE                COMMAND             CREATED          STATUS                      PORTS     NAMES
9c6a0cc84876   webserver:version1   "httpd -k start"    12 seconds ago   Exited (0) 11 seconds ago             webcontainer1
f7795f10d0ab   hello-world          "/hello"            2 weeks ago      Exited (0) 2 weeks ago                naughty_borg
darkdiva@localhost:~/webserver_apache$
```

Figure 9. Build of version 1 and the container that exits at once.

The reason is that `httpd -k start` launches Apache as a daemon: the main process starts the server in the background and then ends with code 0. That process is the main process of the container, so when it ends, the container stops, even though Apache started correctly.

3.2 Version 2: Apache in the foreground. I changed only the last line to `CMD ["httpd", "-D", "FOREGROUND"]`. The build took 0.7 s because steps 2 to 4 are all `CACHED`: only the `CMD` changed. When I ran `webcontainer2`, my terminal stayed blocked, because Apache now runs in the foreground and keeps the container alive. From a second terminal I stopped it with `docker stop webcontainer2`. `docker ps -a` shows it as `Exited (0)` after about two minutes of life, against a fraction of a second for `webcontainer1`.

```
darkdiva@localhost:~/webserver_apache$ sed -i 's/^CMD .*/CMD ["httpd", "-D", "FOREGROUND"]/' Dockerfile
darkdiva@localhost:~/webserver_apache$ tail -n 2 Dockerfile
#Démarrer Apache
CMD ["httpd", "-D", "FOREGROUND"]
darkdiva@localhost:~/webserver_apache$ docker build -t webserver:version2 .
[+] Building 0.7s (9/9) FINISHED                                                                                    docker:default
 => [internal] load build definition from Dockerfile                                                                         0.0s
 => => transferring dockerfile: 369B                                                                                         0.0s
 => [internal] load metadata for registry.access.redhat.com/ubi9/ubi:latest                                                  0.0s
 => [internal] load .dockerignore                                                                                            0.0s
 => => transferring context: 2B                                                                                              0.0s
 => [1/4] FROM registry.access.redhat.com/ubi9/ubi:latest@sha256:16e9e43d99e181463513cc84a514859b1abb9460f1c4d9e208da4b4e51cd5  0.1s
 => => resolve registry.access.redhat.com/ubi9/ubi:latest@sha256:16e9e43d99e181463513cc84a514859b1abb9460f1c4d9e208da4b4e51cd5  0.1s
 => [internal] load build context                                                                                            0.0s
 => => transferring context: 90B                                                                                             0.0s
 => CACHED [2/4] RUN dnf -y install httpd && dnf clean all                                                                   0.0s
 => CACHED [3/4] WORKDIR /var/www/html                                                                                       0.0s
 => CACHED [4/4] COPY index.html .                                                                                           0.0s
 => exporting to image                                                                                                       0.2s
 => => exporting layers                                                                                                      0.0s
 => => exporting manifest sha256:7f17210db7f2ba9f6031fdea23cbe587a06ffd4b097197018c02a0a3a668d526                            0.0s
 => => exporting config sha256:8ae7c13559f890719a2112a5ba1f674575215ae857c67bb8fe4cdf3a578cc68f                              0.0s
 => => exporting attestation manifest sha256:dfbabb8db59878a8fa5b17e6cd54ec1381768501fb925fc5ff8c4293dc2e0cb9                0.0s
 => => exporting manifest list sha256:c73492b48a594f7c4dac2065e9062c0012b03d07a026dd36311cf39943fd5d61                       0.0s
 => => naming to docker.io/library/webserver:version2                                                                        0.0s
 => => unpacking to docker.io/library/webserver:version2                                                                     0.0s
darkdiva@localhost:~/webserver_apache$ docker run --name webcontainer2 webserver:version2
AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.17.0.2. Set the 'ServerName' directi
ve globally to suppress this message
```

Figure 10. Version 2: cached build, and the terminal blocked by Apache in the foreground.



Figure 11. The second terminal: docker stop and docker ps -a.

3.3 Version 3: removing the AH00558 warning. Both runs printed `AH00558: Could not reliably determine the server's fully qualified domain name`. Apache looks for its own name at startup, finds none in the container, and guesses from the IP address. I added a `ServerName localhost` line to the Apache configuration:

```
FROM registry.access.redhat.com/ubi9/ubi
RUN dnf -y install httpd && dnf clean all
# Configuration: nom du serveur
RUN echo "ServerName localhost" >> /etc/httpd/conf/httpd.conf
WORKDIR /var/www/html
COPY index.html .
EXPOSE 80
CMD ["httpd", "-D", "FOREGROUND"]
```

```
darkdiva@localhost:~/webserver_apache$ cat > Dockerfile << 'EOF'
> FROM registry.access.redhat.com/ubi9/ubi
> RUN dnf -y install httpd && dnf clean all
> # Configuration: nom du serveur
> RUN echo "ServerName localhost" >> /etc/httpd/conf/httpd.conf
> WORKDIR /var/www/html
> COPY index.html .
> EXPOSE 80
> CMD ["httpd", "-D", "FOREGROUND"]
> EOF
darkdiva@localhost:~/webserver_apache$ cat Dockerfile
FROM registry.access.redhat.com/ubi9/ubi
RUN dnf -y install httpd && dnf clean all
# Configuration: nom du serveur
RUN echo "ServerName localhost" >> /etc/httpd/conf/httpd.conf
WORKDIR /var/www/html
COPY index.html .
EXPOSE 80
CMD ["httpd", "-D", "FOREGROUND"]
```

Figure 12. Dockerfile of version 3 (note >> to append, not > which would overwrite the file).

```
darkdiva@localhost:~/webserver_apache$ docker build -t webserver:version3 .
[+] Building 1.6s (10/10) FINISHED                                                                                  docker:default
 => [internal] load build definition from Dockerfile                                                                          0.0s
 => => transferring dockerfile: 360B                                                                                          0.0s
 => [internal] load metadata for registry.access.redhat.com/ubi9/ubi:latest                                                   0.1s
 => [internal] load .dockerignore                                                                                             0.0s
 => => transferring context: 2B                                                                                               0.0s
 => [1/5] FROM registry.access.redhat.com/ubi9/ubi:latest@sha256:16e9e43d99e181463513cc84a514859b1abb9460f1c4d9e208da4b4e51cd5  0.1s
 => => resolve registry.access.redhat.com/ubi9/ubi:latest@sha256:16e9e43d99e181463513cc84a514859b1abb9460f1c4d9e208da4b4e51cd5  0.0s
 => [internal] load build context                                                                                             0.0s
 => => transferring context: 90B                                                                                              0.0s
 => CACHED [2/5] RUN dnf -y install httpd && dnf clean all                                                                    0.0s
 => [3/5] RUN echo "ServerName localhost" >> /etc/httpd/conf/httpd.conf                                                       0.3s
 => [4/5] WORKDIR /var/www/html                                                                                               0.1s
 => [5/5] COPY index.html .                                                                                                   0.1s
 => exporting to image                                                                                                        0.7s
 => => exporting layers                                                                                                       0.3s
 => => exporting manifest sha256:41dc3e975baaef697527d3e7efa390b706a5694f7e6fd7d37f22cc391c7e23e0                              0.0s
 => => exporting config sha256:f91b9f1a80fa208590eb43618a34e07ee7a3e7c1b6e83b0733cd16691e05e8d6                                0.0s
 => => exporting attestation manifest sha256:d1ed944c23fe493f2a7a1c02fd9ab039d1cc0ebba039caac547c2acffd30b992                  0.0s
 => => exporting manifest list sha256:0b5cd12214db8356d857010c89b536df58daa8488986eadb311d192d83947123                        0.0s
 => => naming to docker.io/library/webserver:version3                                                                         0.0s
 => => unpacking to docker.io/library/webserver:version3                                                                      0.2s
```

Figure 13. Build of version 3: the new RUN echo step runs, the dnf layer is cached.

I ran it in the background with `docker run -d --name webcontainer3 webserver:version3`. `docker logs webcontainer3` prints **nothing**: the warning is gone. `docker ps` shows the container Up with port `80/tcp`.

> **Problem I ran into**
>
> My first `docker run -d --name webcontainer3` failed with `The container name "/webcontainer3" is already in use`. A stopped container keeps its name until it is removed, and I had already created one with that name. `docker ps -a` showed it as `Exited`, so I removed it with `docker rm -f webcontainer3` and ran it again.

```
darkdiva@localhost:~/webserver_apache$ docker run -d --name webcontainer3 webserver:version3
f5f987a289812877611cc417e72c535c9fa43aeafb13e55344a68fa298299585
darkdiva@localhost:~/webserver_apache$ docker logs webcontainer3
darkdiva@localhost:~/webserver_apache$ docker ps
CONTAINER ID   IMAGE               COMMAND                  CREATED          STATUS          PORTS     NAMES
f5f987a28981   webserver:version3   "httpd -D FOREGROUND"   18 seconds ago   Up 17 seconds   80/tcp    webcontainer3
darkdiva@localhost:~/webserver_apache$
```

Figure 14. webcontainer3 in the background: empty logs, and Up in docker ps.

4. Part 3: reaching the application

4.1 By the IP address of the container. `docker inspect` gives the address of `webcontainer3` on the Docker network: `172.17.0.2`. From the VM, `curl http://172.17.0.2` prints my page. From the

browser of my PC, the same address ends with **`ERR_CONNECTION_TIMED_OUT`**: `172.17.x.x` is a private network that exists only inside the VM, so my PC has no route to it.

```
darkdiva@localhost:~/webserver_apache$ docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' webcontainer3
172.17.0.2
darkdiva@localhost:~/webserver_apache$ curl http://172.17.0.2
'
<!DOCTYPE html>
<html>
   <body><h1>Hello From Docker LAB!</h1></body>
</html>
darkdiva@localhost:~/webserver_apache$
```

Figure 15. The container address and curl from the VM.



Figure 16. The same address from the browser of my PC: timed out.

4.2 Method 1: publishing a port with -p. I stopped `webcontainer3` and ran `docker run -d -p 3000:80 --name webcontainer4 webserver:version3`. The option `-p 3000:80` redirects port 3000 of the VM to port 80 of the container. `curl http://localhost:3000` prints the page, and `docker ps` shows `0.0.0.0:3000->80/tcp`.

```
darkdiva@localhost:~/webserver_apache$ docker stop webcontainer3
webcontainer3
darkdiva@localhost:~/webserver_apache$ docker run -d -p 3000:80 --name webcontainer4 webserver:version3
9b6ba6520b640e30be5c954e7b9bdc6c56df23e46ee63ec1870fc25b8c4258e0
darkdiva@localhost:~/webserver_apache$ curl http://localhost:3000
'
<!DOCTYPE html>
<html>
   <body><h1>Hello From Docker LAB!</h1></body>
</html>
darkdiva@localhost:~/webserver_apache$ docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED          STATUS          PORTS
  NAMES
9b6ba6520b64   webserver:version3   "httpd -D FOREGROUND"   18 seconds ago   Up 17 seconds   0.0.0.0:3000->80/tcp, [::]:3000->80/tcp
  webcontainer4
darkdiva@localhost:~/webserver_apache$
```

Figure 17. Running with -p 3000:80, curl and docker ps.

To open it from Windows, my VM is behind VirtualBox's NAT, so I needed a port forwarding rule (host 127.0.0.1 port 3000 to guest 10.0.2.15 port 3000) and an open port in firewalld. Before the rule, the browser showed `ERR_CONNECTION_REFUSED`; after it, the page appears.

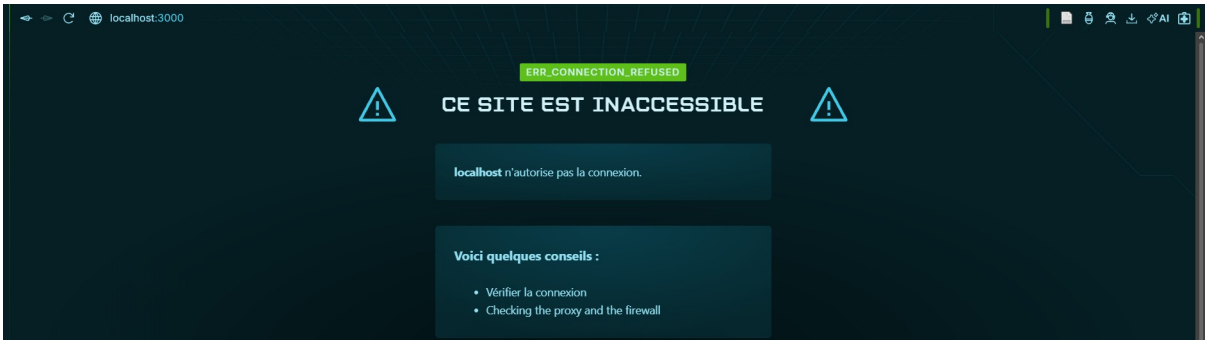


Figure 18. localhost:3000 before the rule: connection refused.

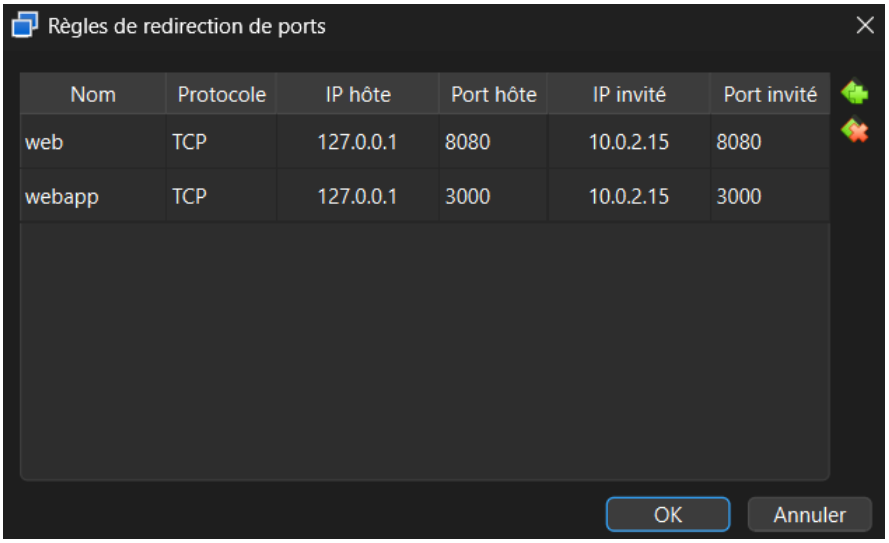


Figure 19. The port forwarding rules in VirtualBox.



Figure 20. The page served by the container, seen from my PC.

4.3 Method 2: the host network. With `--network host`, the container has no network of its own and Apache listens directly on port 80 of the VM. I first tested what happens if Apache is still running on the VM: I started `httpd` and launched `docker run -d --network host --name conflict webserver:version3`. The container is `Exited (1)` and its logs say `(98)Address already in use: AH00072: make_sock: could not bind to address 0.0.0.0:80` and `no listening sockets available, shutting down`. Two Apaches cannot use port 80 at the same time.

> **Problem I ran into**
>
> In this experiment I made typing mistakes: `suddo` instead of `sudo`, and a `run` command without `docker` in front. Bash answered `command not found` both times and showed `Similar command is: sudo`. I retyped them correctly. Later I also typed `sudo systemctlstatus` without a space and got the same kind of error.

```
darkdiva@localhost:~/webserver_apache$ docker stop webcontainer4
webcontainer4
darkdiva@localhost:~/webserver_apache$ suddo systemctl start httpd
bash: suddo: command not found...
Similar command is: 'sudo'
darkdiva@localhost:~/webserver_apache$ sudo systemctl start httpd
[sudo] password for darkdiva:
darkdiva@localhost:~/webserver_apache$ run -d --network host --name conflict webserver:version3
bash: run: command not found...
darkdiva@localhost:~/webserver_apache$ docker run -d --network host --name conflict webserver:version3
f52cb4df5cc3fb619657d0240fceec507b90d46746593b1333e1861e025959f7
darkdiva@localhost:~/webserver_apache$ docker ps -a
CONTAINER ID   IMAGE               COMMAND                  CREATED          STATUS                          PORTS     NAMES
f52cb4df5cc3   webserver:version3  "httpd -D FOREGROUND"    7 seconds ago    Exited (1) 6 seconds ago                  conflict
9b6ba6520b64   webserver:version3  "httpd -D FOREGROUND"    11 minutes ago   Exited (0) About a minute ago             webcontainer4
f5f987a28981   webserver:version3  "httpd -D FOREGROUND"    24 minutes ago   Exited (0) 13 minutes ago                 webcontainer3
0f22bbad439c   webserver:version2  "httpd -D FOREGROUND"    35 minutes ago   Exited (0) 33 minutes ago                 webcontainer2
9c6a0cc84876   webserver:version1  "httpd -k start"         39 minutes ago   Exited (0) 39 minutes ago                 webcontainer1
f7795f10d0ab   hello-world         "/hello"                 2 weeks ago      Exited (0) 2 weeks ago                    naughty_borg
darkdiva@localhost:~/webserver_apache$ docker logs conflict
(98)Address already in use: AH00072: make_sock: could not bind to address [::]:80
(98)Address already in use: AH00072: make_sock: could not bind to address 0.0.0.0:80
no listening sockets available, shutting down
AH00015: Unable to open logs
darkdiva@localhost:~/webserver_apache$
```

Figure 21. The conflict: the container exits with code 1 because port 80 is taken.

After removing `conflict` and stopping Apache again (`inactive (dead)`), I ran `docker run -d --network host --name webcontainer5 webserver:version3`. `curl http://localhost:80` prints my page, and in `docker ps` the **PORTS column is empty**: with the host network there is nothing to publish, so `-p` makes no sense.

```
darkdiva@localhost:~/webserver_apache$ docker rm -f conflict
conflict
darkdiva@localhost:~/webserver_apache$ sudo systemctl disable --now httpd
darkdiva@localhost:~/webserver_apache$ sudo systemctlstatus httpd --no-pager
sudo: systemctlstatus: command not found
darkdiva@localhost:~/webserver_apache$ sudo systemctl status httpd --no-pager
○ httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
     Active: inactive (dead)
       Docs: man:httpd.service(8)

Nov 07 11:27:33 localhost.localdomain systemd[1]: httpd.service: Deactivated successfully.
Nov 07 11:27:33 localhost.localdomain systemd[1]: Stopped httpd.service - The Apache HTTP Server.
Nov 07 12:24:10 localhost.localdomain systemd[1]: Starting httpd.service - The Apache HTTP Server...
Nov 07 12:24:10 localhost.localdomain (httpd)[17115]: httpd.service: Referenced but unset environment variable evaluates to … OPTIONS
Nov 07 12:24:10 localhost.localdomain httpd[17115]: AH00558: httpd: Could not reliably determine the server's fully qualifie… message
Nov 07 12:24:10 localhost.localdomain systemd[1]: Started httpd.service - The Apache HTTP Server.
Nov 07 12:24:10 localhost.localdomain httpd[17115]: Server configured, listening on: port 80
Nov 07 12:26:32 localhost.localdomain systemd[1]: Stopping httpd.service - The Apache HTTP Server...
Nov 07 12:26:33 localhost.localdomain systemd[1]: httpd.service: Deactivated successfully.
Nov 07 12:26:33 localhost.localdomain systemd[1]: Stopped httpd.service - The Apache HTTP Server.
Hint: Some lines were ellipsized, use -l to show in full.
darkdiva@localhost:~/webserver_apache$ docker run -d --network host --name webcontainer5 webserver:version3
f43c74d68087fbcde427f412edfd3fe194e4e5629bd193cb198d899a741185c2
cdarkdiva@localhost:~/webserver_apache$ curl http://localhost:80
'
<!DOCTYPE html>
<html>
   <body><h1>Hello From Docker LAB!</h1></body>
</html>
darkdiva@localhost:~/webserver_apache$ docker ps
CONTAINER ID   IMAGE               COMMAND                  CREATED          STATUS          PORTS     NAMES
f43c74d68087   webserver:version3  "httpd -D FOREGROUND"    23 seconds ago   Up 22 seconds             webcontainer5
darkdiva@localhost:~/webserver_apache$
```

Figure 22. Apache stopped on the VM, then webcontainer5 on the host network.

4.4 Comparing the three ways.

Method	Who can reach it	Main limit	Command
Container IP	Only the VM itself (private Docker network)	The address can change at each start; not reachable from the PC	curl http://172.17.0.2
Published port (-p)	The VM, and the PC with a forwarding rule and the firewall open	One host port per container; the port must be free	-p 3000:80
Host network	The VM on its own port 80, no mapping	No isolation; port 80 must be free on the VM	--network host

5. Part 4: a Flask application

In `~/webserver_python` I wrote `app.py`, a small Flask application, and a Dockerfile based on `python:3.12-slim`:

```
# app.py
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello From Docker Python Web App!"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

```
FROM python:3.12-slim
WORKDIR /app
RUN pip install --no-cache-dir Flask
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

Problem I ran into

The container did not stay up. `docker run` printed an ID, but `curl` answered `Could not connect to server` and `docker ps -a` showed `flask` as `Exited (1)`. `docker logs flask` explained it: `IndentationError: expected an indented block after function definition on line 5`. The `return` line was at the left edge, so Python found no body for `def hello():`. The indentation was lost when I pasted the file into the terminal. I rewrote the file in `nano`, and because `app.py` is copied into the image at build time, I had to rebuild the image: this time the `COPY app.py .` step was not cached.

I also had a warning in the first build: `ConsistentInstructionCasing`, because I had written `Copy` instead of `COPY`. It still worked, but I corrected it. Later I typed `docker -d -p ...` without `run` twice and got `unknown shorthand flag: 'd' in -d`.

```
darkdiva@localhost:~/webserver_apache$ mkdir -p ~/webserver_python && cd ~/webserver_python
darkdiva@localhost:~/webserver_python$ cat > app.py << 'EOF'
> # app.py
> from flask import Flask
> app = Flask(__name__)
> @app.route('/')
> def hello():
> return "Hello From Docker Python Web App!"
> if __name__ == '__main__':
>     app.run(host='0.0.0.0', port=5000)
> EOF
darkdiva@localhost:~/webserver_python$ grep -n "__" app.py
3:app = Flask(__name__)
7:if __name__ == '__main__':
darkdiva@localhost:~/webserver_python$ cat -A app.py | head -12
# app.py$
from flask import Flask$
app = Flask(__name__)$
@app.route('/')$
def hello():$
return "Hello From Docker Python Web App!"$
if __name__ == '__main__':$
    app.run(host='0.0.0.0', port=5000)$
darkdiva@localhost:~/webserver_python$ sed -i 's/^app= Flask(.*)$/app = Flask(__name__)/' app.py
darkdiva@localhost:~/webserver_python$ grep -n "Flask(" app.py
3:app = Flask(__name__)
darkdiva@localhost:~/webserver_python$ cat > Dockerfile << 'EOF'
> FROM python:3.12-slim
> WORKDIR /app
> RUN pip install --no-cache-dir Flask
> Copy app.py .
> EXPOSE 5000
> CMD ["python", "app.py"]
> EOF
darkdiva@localhost:~/webserver_python$ cat Dockerfile
FROM python:3.12-slim
WORKDIR /app
RUN pip install --no-cache-dir Flask
Copy app.py .
EXPOSE 5000
CMD ["python", "app.py"]
darkdiva@localhost:~/webserver_python$ docker build -t webserver:flask .
[+] Building 23.6s (9/9) FINISHED                                                
```

```
 => WARN: ConsistentInstructionCasing: Command 'Copy' should match the case of the command majority (uppercase) (line 4)        0.1s
 => [internal] load metadata for docker.io/library/python:3.12-slim                                                            1.4s
 => [internal] load .dockerignore                                                                                              0.0s
 => => transferring context: 2B                                                                                                0.0s
 => [1/4] FROM docker.io/library/python:3.12-slim@sha256:4840d514c583a4d3d8cad3940e68dee6f0f6f3635c12fcd0a3d74c21a5b93a54      14.1s
 => => resolve docker.io/library/python:3.12-slim@sha256:4840d514c583a4d3d8cad3940e68dee6f0f6f3635c12fcd0a3d74c21a5b93a54       0.1s
 => => sha256:05e0eee5572e9c9f50408a2132590133b830a668a1cf2ff2b150f19c57707e2b 248B / 248B                                     0.1s
 => => sha256:2dc99d2d0a39fce31e37615eb4c932e16dcc4c2b0ce9993714280081b5dbe085 12.12MB / 12.12MB                               3.4s
 => => sha256:68bced9e0afa4479df17c3646e27c7c7e9ce9ba70c0e6c73b6826487b89037d9 1.29MB / 1.29MB                                 1.4s
 => => sha256:37fb86c2a767b88d500c6872195f88078f64765fe0d2f9cc375cc2efbd4c0a41 29.86MB / 29.86MB                               11.2s
 => => extracting sha256:37fb86c2a767b88d500c6872195f88078f64765fe0d2f9cc375cc2efbd4c0a41                                       1.6s
 => => extracting sha256:68bced9e0afa4479df17c3646e27c7c7e9ce9ba70c0e6c73b6826487b89037d9                                      0.2s
 => => extracting sha256:2dc99d2d0a39fce31e37615eb4c932e16dcc4c2b0ce9993714280081b5dbe085                                      0.7s
 => => extracting sha256:05e0eee5572e9c9f50408a2132590133b830a668a1cf2ff2b150f19c57707e2b                                      0.0s
 => [internal] load build context                                                                                              0.1s
 => => transferring context: 286B                                                                                              0.0s
 => [2/4] WORKDIR /app                                                                                                         0.6s
 => [3/4] RUN pip install --no-cache-dir Flask                                                                                 3.6s
 => [4/4] COPY app.py .                                                                                                        0.1s
 => exporting to image                                                                                                         3.4s
 => => exporting layers                                                                                                        0.8s
 => => exporting manifest sha256:1e4b0bfd17a62259c73fc58132e4d124d60ed8e59a1c682c4d54a2ccd9496e04                              0.0s
 => => exporting config sha256:c900e5fbb810322f42d6d5dc14e43c30532afcc785918353cbf0ea6ea435eb7a                                0.0s
 => => exporting attestation manifest sha256:c6529092a8446761dec60410495373376f6b6d01235b1d1a76f10e1ab0c13ff7                  0.0s
 => => exporting manifest list sha256:d13b9471dcf12617d192e365bca3fee5ce4b36e873480b46c800dba258b70789                         0.0s
 => => naming to docker.io/library/webserver:flask                                                                             0.0s
 => => unpacking to docker.io/library/webserver:flask                                                                          2.3s

 1 warning found (use docker --debug to expand):
 - ConsistentInstructionCasing: Command 'Copy' should match the case of the command majority (uppercase) (line 4)
darkdiva@localhost:~/webserver_python$ nano Dockerfile
darkdiva@localhost:~/webserver_python$ docker build -t webserver:flask .
[+] Building 0.9s (9/9) FINISHED                                                                                  docker:default
 => [internal] load build definition from Dockerfile                                                                           0.0s
 => => transferring dockerfile: 219B                                                                                           0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim                                                            0.4s
 => [internal] load .dockerignore                                                                                              0.0s
 => => transferring context: 2B                                                                                                0.0s
 => [1/4] FROM docker.io/library/python:3.12-slim@sha256:4840d514c583a4d3d8cad3940e68dee6f0f6f3635c12fcd0a3d74c21a5b93a54      0.1s
 => => resolve docker.io/library/python:3.12-slim@sha256:4840d514c583a4d3d8cad3940e68dee6f0f6f3635c12fcd0a3d74c21a5b93a54       0.1s
 => [internal] load build context                                                                                              0.0s
 => => transferring context: 87B                                                                                               0.0s
 => CACHED [2/4] WORKDIR /app                                                                                                  0.0s
 => CACHED [3/4] RUN pip install --no-cache-dir Flask                                                                          0.0s
 => CACHED [4/4] COPY app.py .                                                                                                 0.0s
 => exporting to image                                                                                                         0.2s
 => => exporting layers                                                                                                        0.0s
```

Figure 24. The first build with the casing warning, then a cached rebuild.

```
 => => exporting layers                                                                                    0.0s
 => => exporting manifest sha256:1e4b0bfd17a62259c73fc58132e4d124d60ed8e59a1c682c4d54a2ccd9496e04          0.0s
 => => exporting config sha256:c900e5fbb810322f42d6d5dc14e43c30532afcc785918353cbf0ea6ea435eb7a            0.0s
 => => exporting attestation manifest sha256:5f11d827a7894b0b427087f7ba94404a9307e75256c976d6abd9e3337593036d  0.0s
 => => exporting manifest list sha256:a642f8f6f239596fdb2dbf9d1550a848095058a54053cd9ac5f97b186ed9ee82     0.0s
 => => naming to docker.io/library/webserver:flask                                                         0.0s
 => => unpacking to docker.io/library/webserver:flask                                                      0.0s
darkdiva@localhost:~/webserver_python$ docker run -d -p 5000:5000 --name flask webserver:flask
52e3ce0fdf632442f1a0d8d955a427963ab05328a5572c9b1133ccec1aede0a8
darkdiva@localhost:~/webserver_python$ curl http://localhost:5000
curl: (7) Failed to connect to localhost port 5000 after 0 ms: Could not connect to server
darkdiva@localhost:~/webserver_python$ docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED          STATUS          PORTS     NAMES
f43c74d68087   webserver:version3   "httpd -D FOREGROUND"    17 minutes ago   Up 17 minutes             webcontainer5
darkdiva@localhost:~/webserver_python$ docker ps -a
CONTAINER ID   IMAGE                COMMAND                  CREATED              STATUS                              PORTS     NAMES
52e3ce0fdf63   webserver:flask      "python app.py"          About a minute ago   Exited (1) About a minute ago               flask
f43c74d68087   webserver:version3   "httpd -D FOREGROUND"    18 minutes ago       Up 18 minutes                               webcontain
er5
9b6ba6520b64   webserver:version3   "httpd -D FOREGROUND"    31 minutes ago       Exited (0) 21 minutes ago                   webcontain
er4
f5f987a28981   webserver:version3   "httpd -D FOREGROUND"    45 minutes ago       Exited (0) 34 minutes ago                   webcontain
er3
0f22bbad439c   webserver:version2   "httpd -D FOREGROUND"    56 minutes ago       Exited (0) 53 minutes ago                   webcontain
er2
9c6a0cc84876   webserver:version1   "httpd -k start"         About an hour ago    Exited (0) About an hour ago                webcontain
er1
f7795f10d0ab   hello-world          "/hello"                 2 weeks ago          Exited (0) 2 weeks ago                      naughty_bo
rg
darkdiva@localhost:~/webserver_python$ docker logs flask
  File "/app/app.py", line 6
    return "Hello From Docker Python Web App!"
    ^^^^^^
IndentationError: expected an indented block after function definition on line 5
darkdiva@localhost:~/webserver_python$ nano webserver_python
darkdiva@localhost:~/webserver_python$ cd ..
darkdiva@localhost:~$ nano webserver_python
darkdiva@localhost:~$ cd webserver_python
darkdiva@localhost:~/webserver_python$ nano app.py
darkdiva@localhost:~/webserver_python$ docker rm -f flask
flask
darkdiva@localhost:~/webserver_python$ docker build -t webserver:flask .
[+] Building 1.6s (9/9) FINISHED                                                                docker:default
 => [internal] load build definition from Dockerfile                                                       0.0s
 => => transferring dockerfile: 219B                                                                       0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim                                        0.7s
 => [internal] load .dockerignore                                                                          0.0s
 => => transferring context: 2B                                                                            0.0s
```

Figure 25. The container that exited, and the IndentationError in the logs.

After the fix, `docker run -d -p 5000:5000 --name flask webserver:flask` works: `docker ps` shows `0.0.0.0:5000->5000/tcp`, `curl http://localhost:5000` prints `Hello From Docker Python Web App!`, and the logs show Flask running on `0.0.0.0:5000` and a `GET /` answered with code 200. Flask warns that this is a development server, which is normal here.

```
 => [1/4] FROM docker.io/library/python:3.12-slim@sha256:4840d514c583a4d3d8cad3940e68dee6f0f6f3635c12fcd0a3d74c21a5b93a54       0.1s
 => => resolve docker.io/library/python:3.12-slim@sha256:4840d514c583a4d3d8cad3940e68dee6f0f6f3635c12fcd0a3d74c21a5b93a54       0.1s
 => [internal] load build context                                                                                            0.0s
 => => transferring context: 291B                                                                                            0.0s
 => CACHED [2/4] WORKDIR /app                                                                                                0.0s
 => CACHED [3/4] RUN pip install --no-cache-dir Flask                                                                        0.0s
 => [4/4] COPY app.py .                                                                                                      0.1s
 => exporting to image                                                                                                       0.5s
 => => exporting layers                                                                                                      0.2s
 => => exporting manifest sha256:59cf45c16c910c21b82e0ca786163593ddc6a502f4085ed76c516dffae2d9f8f                            0.0s
 => => exporting config sha256:9445b933245153fdd92dc25034aa7f22b049b5419aa10adad4f1ba2533450876                              0.0s
 => => exporting attestation manifest sha256:51be2c9f3086b1bb581f5a8b564008c070b05e0bc124771e29e584203c1b0e45                0.1s
 => => exporting manifest list sha256:1823a91a49e6d10f0fdc8795225f94453aa51e8753ce89e74d6a495649fb8a1c                       0.0s
 => => naming to docker.io/library/webserver:flask                                                                           0.0s
 => => unpacking to docker.io/library/webserver:flask                                                                        0.0s
darkdiva@localhost:~/webserver_python$ docker -d -p 5000:5000 --name flask webserver:flask
unknown shorthand flag: 'd' in -d

Usage:  docker [OPTIONS] COMMAND [ARG...]

Run 'docker --help' for more information
darkdiva@localhost:~/webserver_python$ docker -d -p  5000:5000 --name flask webserver:flask
unknown shorthand flag: 'd' in -d

Usage:  docker [OPTIONS] COMMAND [ARG...]

Run 'docker --help' for more information
darkdiva@localhost:~/webserver_python$ docker run -d -p 5000:5000 --name flask webserver:flask
8bce715d1375380c682be100b80bdf878fb397043c81e1a43bec116cbbcda66f
darkdiva@localhost:~/webserver_python$ docker ps
CONTAINER ID   IMAGE                 COMMAND                  CREATED          STATUS          PORTS
      NAMES
8bce715d1375   webserver:flask       "python app.py"          28 seconds ago   Up 27 seconds   0.0.0.0:5000->5000/tcp, [::]:5000->5000/
tcp   flask
f43c74d68087   webserver:version3    "httpd -D FOREGROUND"    23 minutes ago   Up 23 minutes
      webcontainer5
darkdiva@localhost:~/webserver_python$ curl http://localhost:5000
Hello From Docker Python Web App!darkdiva@localhost:~/webserver_python$ docker logs flask
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [07/Nov/2026 11:51:39] "GET / HTTP/1.1" 200 -
darkdiva@localhost:~/webserver_python$
```

Figure 26. Rebuild, run, curl and the Flask logs.

Access from my PC. I opened port 5000 in firewalld (my first command had a typo, `--add-ports`, which `firewall-cmd` does not know; the option is `--add-port`) and added a port forwarding rule in VirtualBox for 127.0.0.1:5000 to 10.0.2.15:5000. The page then loads in the browser of my PC.

```
darkdiva@localhost:~/webserver_python$ sudo firewall-cmd --permanent --add-ports=5000/tcp
[sudo] password for darkdiva:
usage: 'firewall-cmd --help' for usage information or see firewall-cmd(1) man page
firewall-cmd: error: unrecognized arguments: --add-ports=5000/tcp
darkdiva@localhost:~/webserver_python$ sudo firewall-cmd --reload
success
darkdiva@localhost:~/webserver_python$
```

Figure 27. The firewall command with the typo, and the reload.

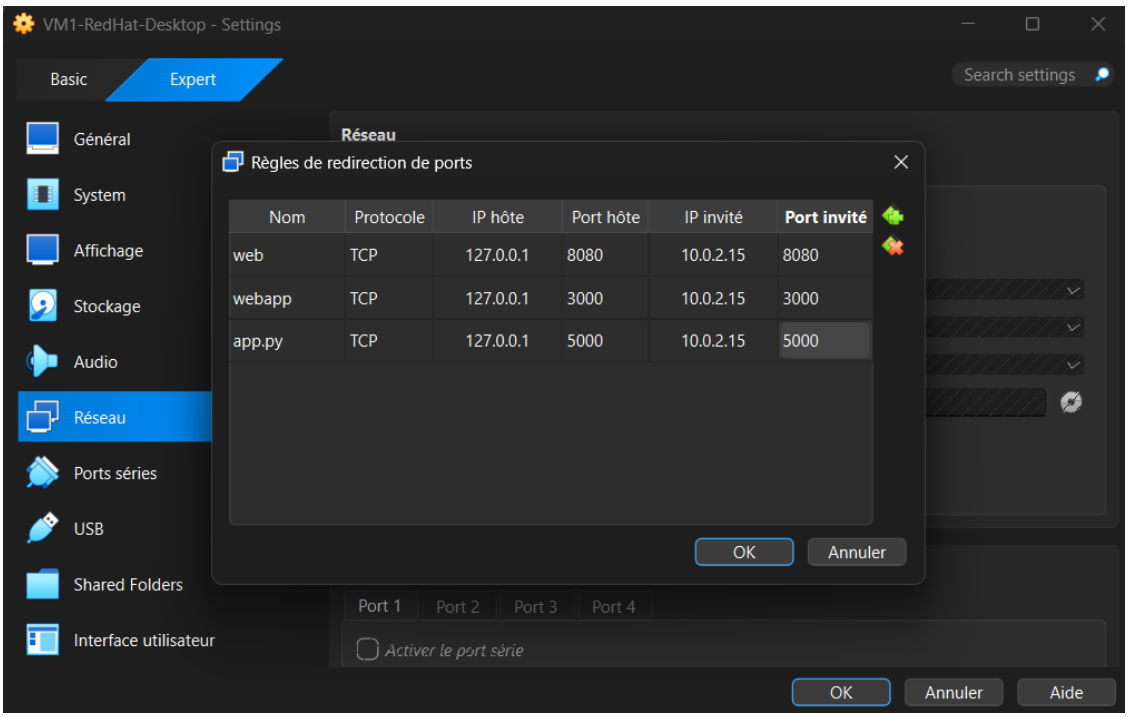


Figure 28. The three forwarding rules: web (8080), webapp (3000) and the Flask application (5000).



Figure 29. The Flask application in the browser of my PC.

6. Cleaning up

Before the cleanup, `docker system df` showed 7 containers (213 kB), 19 images (3.656 GB) and 2.947 GB of build cache. I removed the containers with `docker rm`. My first command removed `webcontainer1` to `webcontainer4`, but it refused `webcontainer5` and `flask` because they were still running ("stop the container before removing or force remove"). I stopped them with `docker stop` and removed them again. Afterwards, only one old exited container remains (from Lab 04), so the containers went from 7 to 1 and the space from 213 kB to 0 B. `docker image prune` freed **0 B**: I had no dangling images. About 3.1 GB of images are still reclaimable, but they are base images that are not dangling, so `prune` leaves them alone.

```
darkdiva@localhost:~/webserver_python$ sudo firewall-cmd --reload
success
darkdiva@localhost:~/webserver_python$ docker system df
TYPE            TOTAL     ACTIVE    SIZE      RECLAIMABLE
Images          19        5         3.656GB   2.911GB (79%)
Containers      7         2         213kB     24.58kB (11%)
Local Volumes   2         0         26.12MB   26.12MB (100%)
Build Cache     76        0         2.947GB   363.5MB
darkdiva@localhost:~/webserver_python$ docker rm webcontainer1 webcontainer2 webcontainer3 webcontainer4 webcontainer5 flask
webcontainer1
webcontainer2
webcontainer3
webcontainer4
Error response from daemon: cannot remove container "webcontainer5": container is running: stop the container before removing or forc
e remove
Error response from daemon: cannot remove container "flask": container is running: stop the container before removing or force remove
darkdiva@localhost:~/webserver_python$ docker stop webcontainer5 flask
webcontainer5
flask
darkdiva@localhost:~/webserver_python$ docker rm webcontainer5 flask
webcontainer5
flask
darkdiva@localhost:~/webserver_python$ docker system df
TYPE            TOTAL     ACTIVE    SIZE      RECLAIMABLE
Images          19        1         3.656GB   3.105GB (84%)
Containers      1         0         0B        0B
Local Volumes   2         0         26.12MB   26.12MB (100%)
Build Cache     76        0         2.947GB   363.5MB
darkdiva@localhost:~/webserver_python$ docker image prune
WARNING! This will remove all dangling images.
Are you sure you want to continue? [y/N] y
Total reclaimed space: 0B
darkdiva@localhost:~/webserver_python$
```

Figure 30. docker system df before and after the cleanup, and docker image prune.

7. Answers to the questions

Which page does Apache show when /var/www/html is empty, with which HTTP code, and why? It shows the Red Hat welcome page with the code **403 Forbidden**. The folder is empty, so there is no `index.html`, and Apache refuses to list the folder (`Options -Indexes`). The file `/etc/httpd/conf.d/welcome.conf` turns that 403 error into the welcome page with `ErrorDocument 403 /.noindex.html`. Once I created my own `index.html`, the code became 200 OK.

Why does webcontainer1 not appear in docker ps? Because it has already stopped. A container lives only as long as its main process runs in the foreground. In version 1, `httpd -k start` starts Apache as a background daemon and the main process ends at once with code 0, so the container stops (`Exited (0)` in `docker ps -a`). In version 2, `httpd -D FOREGROUND` keeps Apache as the main process, so the container stays up.

What does the AH00558 warning mean, and how does version 3 remove it? Apache could not find the fully qualified domain name of the server at startup, so it guessed from the container's IP address (172.17.0.2). The warning is harmless. In version 3 I append `ServerName localhost` to `/etc/httpd/conf/httpd.conf` with a `RUN echo ... >>` instruction, so Apache knows its name and the logs are empty.

What is the difference between the three ways of reaching the application? With the container IP, the address belongs to a private Docker network, so only the VM can use it and my PC times out. With `-p 3000:80`, a port of the VM is redirected to port 80 of the container, so the application is reachable on the VM's address (and on my PC with a forwarding rule and the firewall open). With `--network host`, the container shares the VM's network and Apache uses port 80 of the VM directly, without isolation and without any `-p`. The table in section 4.4 sums it up.

Why must httpd be stopped on the VM before running a container with --network host? Because both Apaches would need port 80 of the same network. My experiment proves it: with `httpd` running on the VM,

the `conflict` container exited with code 1 and its logs said `Address already in use` and `could not bind to address 0.0.0.0:80`. Once I stopped the VM's Apache, `webcontainer5` started normally.

Why does the Flask application use host='0.0.0.0'? It makes Flask listen on all the network interfaces of the container. With `127.0.0.1`, Flask would only listen on the container's own loopback, and the traffic coming from `-p 5000:5000` arrives on the container's network interface, so it would never reach the application, even with the port published.

How can the Flask application be reached from another machine? The application listens on 0.0.0.0, the port is published with `-p 5000:5000`, port 5000 is open in firewalld (`firewall-cmd --permanent --add-port=5000/tcp` then `--reload`), and the VM must be reachable: with NAT, by a port forwarding rule from the host to 10.0.2.15:5000; with a bridged adapter, by using the VM's own address. I tested it with the forwarding rule and `http://localhost:5000` in my browser.

How much space did the cleanup free? The containers went from 7 (213 kB) to 1 (0 B), so about 213 kB were freed. `docker image prune` freed 0 B because I had no dangling images. About 3.1 GB of unused base images remain reclaimable, but `prune` does not remove them.

8. Summary

Task	Result	Comment
Part 1: Apache on the VM	Done	403 with the welcome page, then 200 OK with my page
Part 2: image version 1	Done	COPY destination missing at first; container exits at once
Part 2: image version 2	Done	Foreground Apache; container stays up until stopped
Part 2: image version 3	Done	ServerName added; no AH00558 warning
Part 3: container IP	Done	Works from the VM, times out from my PC
Part 3: published port (-p)	Done	Port 3000 with a forwarding rule and the firewall open
Part 3: host network	Done	Port conflict shown, then works with Apache stopped
Part 4: Flask application	Done	IndentationError fixed; page visible on my PC
Cleanup	Done	7 containers down to 1; prune freed 0 B

9. What I learned

The most useful lesson was the life of a container: it lasts exactly as long as its main process stays in the foreground. The same Apache gave a container that disappeared at once (`httpd -k start`) and a container that stayed up (`httpd -D FOREGROUND`), and only the last line of the Dockerfile changed. It also showed me the cache: the version 2 build took 0.7 seconds because only the `CMD` was different.

The three ways of reaching the application helped me understand Docker networking. The container's own address is private, a published port opens a door from the VM to the container, and the host network

removes the isolation, which is why the port conflict appears. For my PC, the VirtualBox NAT is one more layer to open, with a forwarding rule.

Many of my errors came from details: the missing destination in `COPY`, a name already used by a stopped container, a missing `run`, a lost indentation in Python. I learned to read the error message and the logs first (`docker logs` and `docker ps -a` were enough every time), and to remember that a file copied into an image only changes after a new build.